NOLTA, IEICE

Paper

On the Study of Memory-Less quasi-Newton Method with Momentum Term for Neural Network Training

Shahrzad Mahboubi [1a)], *Ryo Yamatomi* [1], *Indrapriyadarsini S* [2], *Hiroshi Ninomiya* [1], *and Hideki Asai* [2]

[1] *Graduate School of Electrical and Information Engineering, Shonan Institute of Technology 1-1-25 Tsujido-Nishikaigan, Fujisawa, Kanagawa, 251-8511, Japan*

[2] *Graduate School of Science and Technology, Shizuoka University 3-5-1 Johoku, Naka-ku, Hamamatsu, Shizuoka, 432-8011, Japan*

[a)] *20T2502@sit.shonan-it.ac.jp*

Received October 27, 20XX; Revised December 29, 20XX; Published July 1, 20XX

Abstract: Quasi-Newton (QN) methods have shown to be effective in training neural networks. However, computation and storage of the approximated Hessian matrix in large-scale applications is still a problem. The Memory-less QN (MLQN) introduced a method that did not require the storage of the matrix. This paper describes the effectiveness of the memory-less scheme, when applied to momentum, accelerated QN methods through computer simulations on function approximation and classification problems.

Key Words: Neural networks, training algorithm, memory-less quasi-Newton method, Nesterov's accelerated gradient, momentum term

1. Introduction

Neural network (NN) techniques are effective in numerous real-world applications such as highly nonlinear function approximation problems, image and natural language processing applications, etc. [1]. Most of these applications require NN models to achieve good accuracy and low errors in minimum time. The choice of the training algorithm is the most critical step in developing NN models.

Gradient-based algorithms are popularly used in NN training and can be categorized broadly into first and second-order methods based on the form of learning rate [2]. First-order methods, especially Adam, which is the most famous algorithm, are often used because each technique uses a scalar as a learning rate and has the simple calculation of the update formula [1]. However, when applied to highly nonlinear function approximation problems, first-order methods still converge too slowly, and optimization error cannot be effectively reduced within finite time despite its advantage [3]. Typically, the second-order method, such as the quasi-Newton method (QN), has been used to deal with this problem [1–4]. In recent years, various improvement studies have been conducted on QN. Nesterov's Accelerated quasi-Newton (NAQ) [3] and Momentum quasi-Newton (MoQ) [2] methods have been introduced to accelerate QN by incorporating momentum term. On the other hand, QN-based methods needed the approximated Hessian of the curvature information as the learning

Nonlinear Theory and Its Applications, IEICE, vol. 2, no. 3, pp. 1101–1106 DOI: 10.1587/nolta.2.1101

rate [1,2]. Consequently, the computational cost was exceptionally high compared to first-order methods. Various studies have been conducted to conquer this problem. One approach is Limited-Memory quasi-Newton (LQN) method [4] which limits the amount of memory of the approximated Hessian. Another is Memory-Less quasi-Newton (MLQN) method. MLQN updates the parameters only by inner products of vectors without storing the matrix. LQN and MLQN are closely related to the nonlinear Conjugate Gradient (CG) method under the exact line search [4, 5].

This paper studies the effectiveness of the memory-less scheme for two types of momentum-based QN methods, NAQ and MoQ. The performances of the proposed algorithms are demonstrated through computer simulations and compared to conventional training methods using several benchmark problems.

Let $\mathbf{w}$ be the weight vectors, the error function $E(\mathbf{w})$ and the k^{th} iterative formula of the optimization algorithms are defined as

$$\min_{\mathbf{w}\in R^n} \quad E(\mathbf{w}) = \frac{1}{|T_r|}\sum_{p\in T_r} E_p(\mathbf{w}) \qquad \text{and} \qquad \mathbf{w}_{k+1} = \mathbf{w}_k + \mathbf{v}_{k+1}, \tag{1}$$

respectively. $E_p(\mathbf{w})$ denotes the error function of the p^{th} training sample, such as the mean squared error or the cross-entropy. T_r and $|T_r|$ are the training data set and the number of training samples, respectively. Gradient-based algorithms have been commonly used for training NNs. The standard Gradient Descent (GD) method has the update vector of $\mathbf{v}_{k+1} = -\eta_k \nabla E(\mathbf{w}_k)$ where η_k and $\nabla E(\mathbf{w}_k)$ denote the scalar learning rate and the gradient vector, respectively.

2. Memory-Less quasi-Newton Method

The quasi-Newton (QN) method is one of the most efficient optimization algorithms [4] and has the update vector given by

$$\mathbf{v}_{k+1} = \eta_k \mathbf{d}_k \;\text{ and }\; \mathbf{d}_k = -\mathbf{H}_k \nabla E(\mathbf{w}_k). \tag{2}$$

The learning rate η_k is determined by Armijo's condition [4]. $\mathbf{H}_k$ denotes the inverse approximated Hessian and is iteratively updated by the following Broyden-Fletcher-Goldfarb-Shanno (BFGS) formula [4],

$$\begin{aligned} \mathbf{H}_{k+1} = \mathbf{H}_k - \left(\left((\mathbf{H}_k\mathbf{y}_k)\mathbf{s}_k^{\mathrm{T}} + \mathbf{s}_k(\mathbf{H}_k\mathbf{y}_k)^{\mathrm{T}}\right)/\mathbf{s}_k^{\mathrm{T}}\mathbf{y}_k\right) + \left(1 + \mathbf{y}_k^{\mathrm{T}}\mathbf{H}_k\mathbf{y}_k/\mathbf{s}_k^{\mathrm{T}}\mathbf{y}_k\right)\left(\mathbf{s}_k\mathbf{s}_k^{\mathrm{T}}/\mathbf{s}_k^{\mathrm{T}}\mathbf{y}_k\right), \\ \mathbf{s}_k = \mathbf{w}_{k+1} - \mathbf{w}_k \quad \text{and} \quad \mathbf{y}_k = \nabla E\left(\mathbf{w}_{k+1}\right) - \nabla E\left(\mathbf{w}_k\right). \end{aligned} \tag{3}$$

The update of $\mathbf{H}_k$ in QN needs the memory to store the matrix. This is the disadvantage of QN for the large-scale NN training. Memory-Less quasi-Newton (MLQN) method was proposed for this problem [5]. In MLQN, the matrix $\mathbf{H}_{k+1}$ was updated by (3) but the previous $\mathbf{H}_k$ was replaced by the scaled identity matrix of $\tau_k\mathbf{I}$. The direction vector $\mathbf{d}_k$ of MLQN is derived as

$$\begin{aligned} \mathbf{d}_k^{\mathrm{MLQN}} &= -\left((\tau_k\mathbf{I}) - \left(\tau_k(\mathbf{y}_k\mathbf{s}_k^{\mathrm{T}} + \mathbf{s}_k\mathbf{y}_k^{\mathrm{T}})/\mathbf{s}_k^{\mathrm{T}}\mathbf{y}_k\right) + \left(1 + \tau_k\mathbf{y}_k^{\mathrm{T}}\mathbf{y}_k/\mathbf{s}_k^{\mathrm{T}}\mathbf{y}_k\right)\left(\mathbf{s}_k\mathbf{s}_k^{\mathrm{T}}/\mathbf{s}_k^{\mathrm{T}}\mathbf{y}_k\right)\right)\mathbf{g}_k^{\mathrm{MLQN}} \\ &= -\tau_k(\mathbf{g}_k^{\mathrm{MLQN}} - (\beta_k\mathbf{y}_k + \psi_k\mathbf{s}_k)) - (1 + \tau_k\omega_k)\beta_k\mathbf{s}_k, \end{aligned} \tag{4}$$

where $\mathbf{g}_k^{\mathrm{MLQN}} = \nabla E(\mathbf{w}_k)$, $\beta_k = \mathbf{s}_k^{\mathrm{T}}\mathbf{g}_k^{\mathrm{MLQN}}/\mathbf{s}_k^{\mathrm{T}}\mathbf{y}_k$, $\psi_k = \mathbf{y}_k^{\mathrm{T}}\mathbf{g}_k^{\mathrm{MLQN}}/\mathbf{s}_k^{\mathrm{T}}\mathbf{y}_k$, $\omega_k = \mathbf{y}_k^{\mathrm{T}}\mathbf{y}_k/\mathbf{s}_k^{\mathrm{T}}\mathbf{y}_k$ and the self-scaling coefficient $\tau_k = \mathbf{s}_k^{\mathrm{T}}\mathbf{y}_k/\mathbf{y}_k^{\mathrm{T}}\mathbf{y}_k$ [6]. From (4) it can be seen that $\mathbf{d}_k$ of MLQN is calculated only by the inner products of vectors without storing the matrix (hence memory-less). That is, the computational cost was reduced to the same order of first-order methods. The algorithm of MLQN is shown in Algorithm 1.

3. Memory-Less quasi-Newton Method with Momentum Term

The Nesterov's Accelerated QN (NAQ) method [3] and Momentum QN (MoQ) method [2] accelerate the standard QN method using the momentum term. In this paper, the memory-less scheme is introduced to both NAQ and MoQ to show the effectiveness of the momentum term for MLQN. Thus two novel training algorithms are proposed as Memory-Less NAQ (MLNAQ) and Memory-Less MoQ (MLMoQ) in this paper.

NAQ was derived from the quadratic approximation of $E(\mathbf{w})$ around $\mathbf{w}_k + \mu_k\mathbf{v}_k$ whereas QN was approximated around $\mathbf{w}_k$ [3]. The update vector of NAQ was defined as

$$\mathbf{v}_{k+1} = \mu_k\mathbf{v}_k + \eta_k\mathbf{d}_k, \;\text{ and }\; \mathbf{d}_k = -\hat{\mathbf{H}}_k\nabla E(\mathbf{w}_k + \mu_k\mathbf{v}_k), \tag{5}$$

Algorithm 1: MLQN

1. $k = 1$
2. Initialize $\mathbf{w}_k$= random$[-0.5, 0.5]$;
3. Calculate $\nabla E(\mathbf{w}_k)$;
4. **While**$(\|\nabla E(\mathbf{w}_k)\| > \epsilon$ and $k < k_{max})$
   (a) Update $\mathbf{d}_k^{\mathrm{MLQN}}$ using (4);
   (b) Calculate learning rate η_k;
   (c) Update $\mathbf{v}_{k+1} = \eta_k \mathbf{d}_k^{\mathrm{MLQN}}$;
   (d) Update $\mathbf{w}_{k+1} = \mathbf{w}_k + \mathbf{v}_{k+1}$;
   (e) Calculate $\nabla E(\mathbf{w}_{k+1})$;
   (f) Update $\mathbf{s}_k$ and $\mathbf{y}_k$;
   (g) $k = k + 1$
4. **return** $\mathbf{w}_k$;

Algorithm 2: MLNAQ

1. $k = 1$
2. Initialize $\mathbf{w}_k$= random$[-0.5, 0.5]$ and $\mathbf{v}_k = \mathbf{0}$;
3. **While**$(\|\nabla E(\mathbf{w}_k)\| > \epsilon$ and $k < k_{max})$
   (a) Calculate $\nabla E(\mathbf{w}_k + \mu_k \mathbf{v}_k)$;
   (b) Update $\mathbf{d}_k^{\mathrm{MLNAQ}}$ using (7);
   (c) Calculate learning rate η_k;
   (d) Update $\mathbf{v}_{k+1} = \mu_k \mathbf{v}_k + \eta_k \mathbf{d}_k^{\mathrm{MLNAQ}}$;
   (e) Update $\mathbf{w}_{k+1} = \mathbf{w}_k + \mathbf{v}_{k+1}$;
   (f) Calculate $\nabla E(\mathbf{w}_{k+1})$;
   (g) Update $\mathbf{p}_k$ and $\mathbf{q}_k$;
   (h) $k = k + 1$
5. **return** $\mathbf{w}_k$;

where $\mu_k \mathbf{v}_k$ is the momentum term and μ_k denotes the momentum coefficient which is adaptively calculated [2]. The matrix $\hat{\mathbf{H}}_{k+1}$ was updated by

$$\begin{gathered}\hat{\mathbf{H}}_{k+1} = \hat{\mathbf{H}}_k - \left(\left((\hat{\mathbf{H}}_k \mathbf{q}_k)\mathbf{p}_k^{\mathrm{T}} + \mathbf{p}_k(\hat{\mathbf{H}}_k \mathbf{q}_k)^{\mathrm{T}}\right)/\mathbf{p}_k^{\mathrm{T}}\mathbf{q}_k\right) + \left((1 + \mathbf{q}_k^{\mathrm{T}}\hat{\mathbf{H}}_k \mathbf{q}_k/\mathbf{p}_k^{\mathrm{T}}\mathbf{q}_k)\left(\mathbf{p}_k\mathbf{p}_k^{\mathrm{T}}/\mathbf{p}_k^{\mathrm{T}}\mathbf{q}_k\right)\right), \\ \mathbf{p}_k = \mathbf{w}_{k+1} - (\mathbf{w}_k + \mu_k \mathbf{v}_k) \quad \text{and} \quad \mathbf{q}_k = \nabla E(\mathbf{w}_{k+1}) - \nabla E(\mathbf{w}_k + \mu_k \mathbf{v}_k).\end{gathered} \tag{6}$$

NAQ required the past matrix to update $\hat{\mathbf{H}}_k$, in the same way as QN. To reduce the computational cost, we also introduce the memory-less technique into NAQ as MLNAQ. The direction $\mathbf{d}_k^{\mathrm{MLNAQ}}$ is derived from

$$\begin{aligned}\mathbf{d}_k^{\mathrm{MLNAQ}} &= -\left((\tau_k \mathbf{I}) - \left(\tau_k(\mathbf{q}_k\mathbf{p}_k^{\mathrm{T}} + \mathbf{p}_k\mathbf{q}_k^{\mathrm{T}})/\mathbf{p}_k^{\mathrm{T}}\mathbf{q}_k\right) + \left(1 + \tau_k \mathbf{q}_k^{\mathrm{T}}\mathbf{q}_k/\mathbf{p}_k^{\mathrm{T}}\mathbf{q}_k\right)\left(\mathbf{p}_k\mathbf{p}_k^{\mathrm{T}}/\mathbf{p}_k^{\mathrm{T}}\mathbf{q}_k\right)\right)\mathbf{g}_k^{\mathrm{MLNAQ}} \\ &= -\tau_k(\mathbf{g}_k^{\mathrm{MLNAQ}} - (\beta_k \mathbf{q}_k + \psi_k \mathbf{p}_k)) - (1 + \tau_k \omega_k)\beta_k \mathbf{p}_k,\end{aligned} \tag{7}$$

where $\mathbf{g}_k^{\mathrm{MLNAQ}} = \nabla E(\mathbf{w}_k + \mu_k \mathbf{v}_k)$, and scalars are calculated by inner products as $\beta_k = \mathbf{p}_k^{\mathrm{T}}\mathbf{g}_k^{\mathrm{MLNAQ}}/\mathbf{p}_k^{\mathrm{T}}\mathbf{q}_k$, $\psi_k = \mathbf{q}_k^{\mathrm{T}}\mathbf{g}_k^{\mathrm{MLNAQ}}/\mathbf{p}_k^{\mathrm{T}}\mathbf{q}_k$, $\omega_k = \mathbf{q}_k^{\mathrm{T}}\mathbf{q}_k/\mathbf{p}_k^{\mathrm{T}}\mathbf{q}_k$ and $\tau_k = \mathbf{p}_k^{\mathrm{T}}\mathbf{q}_k/\mathbf{q}_k^{\mathrm{T}}\mathbf{q}_k$. The algorithm of MLNAQ is shown in Algorithm 2. From the algorithm, it can be seen that MLNAQ needs to calculate two types of gradients - the Nesterov's accelerated gradient $\nabla E(\mathbf{w}_k + \mu_k \mathbf{v}_k)$ and normal gradient $\nabla E(\mathbf{w}_k)$ in each iteration as shown in steps 3(a) and 3(f). This is NAQ's inherent disadvantage.

MoQ was proposed to resolve the above problem. The Nesterov's accelerated gradient $\nabla E(\mathbf{w}_k + \mu_k \mathbf{v}_k)$ was approximated as a linear combination of normal gradients as shown in (8) by assuming that $E(\mathbf{w})$ is approximately quadratic function [2].

$$\nabla E(\mathbf{w}_k + \mu_k \mathbf{v}_k) \simeq \nabla E(\mathbf{w}_k) + \mu_k \nabla E(\mathbf{v}_k) = (1 + \mu_k)\nabla E(\mathbf{w}_k) - \mu_k \nabla E(\mathbf{w}_{k-1}). \tag{8}$$

The update vector of MoQ is given by

$$\mathbf{v}_{k+1} = \mu_k \mathbf{v}_k + \eta_k \mathbf{d}_k, \quad \text{and} \quad \mathbf{d}_k = -\hat{\mathbf{H}}_k\{(1 + \mu_k)\nabla E(\mathbf{w}_k) - \mu_k \nabla E(\mathbf{w}_{k-1})\}. \tag{9}$$

Here, we consider to implement the memory-less techniqe into MoQ as MLMoQ to reduce the computational cost of MLNAQ. The direction vector of MLMoQ is derived by replacing $\mathbf{q}_k$ and $\mathbf{g}_k^{\mathrm{MLNAQ}}$ in (7) with $\hat{\mathbf{q}}_k$ and $\mathbf{g}_k^{\mathrm{MLMoQ}}$ as shown in (10) below.

$$\hat{\mathbf{q}}_k = \nabla E(\mathbf{w}_{k+1}) - ((1 + \mu_k)\nabla E(\mathbf{w}_k) - \mu_k \nabla E(\mathbf{w}_{k-1})) \quad \text{and} \quad \mathbf{g}_k^{\mathrm{MLMoQ}} = (1 + \mu_k)\nabla E(\mathbf{w}_k) - \mu_k \nabla E(\mathbf{w}_{k-1}). \tag{10}$$

Thus, the direction of MLMoQ is given as

$$\mathbf{d}_k^{\mathrm{MLMoQ}} = -\tau_k(\mathbf{g}_k^{\mathrm{MLMoQ}} - (\beta_k \hat{\mathbf{q}}_k + \psi_k \mathbf{p}_k)) - (1 + \tau_k \omega_k)\beta_k \mathbf{p}_k. \tag{11}$$

The algorithm of MLMoQ is shown in Algorithm 3. This algorithm shows that MLMoQ computes the gradient only once per iteration in step 4(e). Note that, the global convergence terms of [2] are introduced in $\mathbf{y}_k$, $\mathbf{q}_k$ and $\hat{\mathbf{q}}_k$ for the numerical stability of MLQN, MLNAQ and MLMoQ.

Algorithm 3: MLMoQ

1. $k = 1$
2. Initialize $\mathbf{w}_k$= random$[-0.5, 0.5]$ and $\mathbf{v}_k = \mathbf{0}$;
3. Calculate $\nabla E(\mathbf{w}_k)$;
4. **While**$(\|\nabla E(\mathbf{w}_k)\| > \epsilon$ and $k < k_{max})$
   (a) Update $\mathbf{d}_k^{\text{MLNAQ}}$ using (11);
   (b) Calculate learning rate η_k;
   (c) Update $\mathbf{v}_{k+1} = \mu_k \mathbf{v}_k + \eta_k \mathbf{d}_k^{\text{MLMoQ}}$;
   (d) Update $\mathbf{w}_{k+1} = \mathbf{w}_k + \mathbf{v}_{k+1}$;
   (e) Calculate $\nabla E(\mathbf{w}_{k+1})$;
   (f) Update $\mathbf{p}_k$ and $\hat{\mathbf{q}}_k$;
   (g) $k = k + 1$
5. **return** $\mathbf{w}_k$;

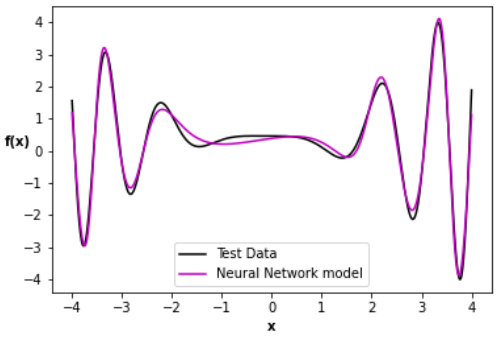

Fig. 1. Comparison between test data of (12) and neural model trained by MLMQN.

4. Simulation results

This section discusses the performance of MLNAQ and MLMoQ by comparing with conventional algorithms such as Adam [1], CG [5], LQN (with memory 1, 2, and 16) [4], and MLQN [6]. Note that, the full-batch strategy is applied to all algorithms in this paper. The hyperparameters of Adam are set to the default values given in [1]. The learning rate η_k for CG, LQN, MLQN, MLNAQ, and MLMoQ are determined by Armijo's conditions [2]. The termination conditions are set to $\epsilon = 10^{-6}$ and $k_{max} = 150,000$ for all problems. The simulations were performed in C with openMP on the AMD Ryzen 7 1700X and 16GB RAM.

Problem 1: function approximation problem

First, (12) is used as a function modeling problem with high nonlinearity [2, 3],

$$f(x) = 1 + (x + 2x^2)\sin(-x^2), \qquad |x| \leq 4. \tag{12}$$

The error function $E(\mathbf{w})$ is given by the mean squared error (MSE) as $E_p(\mathbf{w}) = \frac{1}{2}\|\mathbf{d}_p - \mathbf{o}_p\|^2$, where $\mathbf{o}_p$ and $\mathbf{d}_p$ are the p^{th} output and desired vectors, respectively. The number of training data is set to $|T_r| = 400$. The trained network has a hidden layer with ten neurons. Therefore, the structure of NN is 1-10-1. Each hidden neuron has a sigmoid activation function as $sig(z) = 1/(1 + exp(-z))$. The median and average of $E(\mathbf{w})$, the average computation time (sec), and the average iteration count (k) for ten trials are shown in Table I. We consider the convergence rates (%) for all algorithms as the percentage of the number of trials in which the sufficiently small errors were obtained by the end of the training. From this table, it can be seen that almost all algorithms converge to similar training errors. Because of the highly nonlinear characteristics of this problem, no algorithm has achieved a 100% convergence rate. Adam, CG, LQN(1) and MLQN require more iterations to converge. However, in LQN, the iteration counts decreases as the memory increases, but the computational cost for one iteration increases. This is clear from the per iteration time of LQN. This property is inherent in LQN for highly nonlinear modeling problems. Therefore, LQN(16) has the least iteration number, but the computational time per iteration is the maximum compared to all algorithms. Then the total time of LQN(16) is larger than LQN(2). On the other hand, in comparing memory-less schemes, MLNAQ and MLMoQ can converge with fewer iterations than MLQN. This means that the momentum term are effective for the MLQN. Furthermore, it can be seen that the time required for one iteration of MLMoQ is almost the same as MLQN,

Table I. Summary of simulation results of (12).

Algorithm	Iteration counts (k)	Time (sec)	Per iteration time(sec)($\times 10^{-3}$)	$E_{train}(\mathbf{w})(\times 10^{-3})$ Med. / Ave.	Convergence rate (%)
Adam	90,908	7.75	0.085	0.999 / 1.028	80
CG	52,636	6.00	0.113	0.996 / 0.995	60
LQN(1)	81,577	10.20	0.125	0.999 / 0.999	80
LQN(2)	39,260	5.39	0.137	0.999 / 0.999	70
LQN(16)	26,138	6.75	0.258	0.999 / 0.999	60
MLQN	80,729	9.79	0.121	0.999 / 0.999	80
MLNAQ	33,493	6.85	0.205	0.999 / 0.999	80
MLMoQ	29,834	**3.65**	0.122	0.999 / 0.999	70

Table II. Summary of 3-Spiral simulation results.

Algorithm	Iteration counts (k)	Time (sec)	Per iteration time(sec)($\times 10^{-3}$)	$E_{train}(\mathbf{w})(\times 10^{-3})$ Med. / Ave.	Convergence rate (%)
Adam	55,038	14.64	0.266	0.840 / 0.838	100
CG	150,000	57.21	0.381	1.378 / 1.381	100
LQN(1)	9,918	3.98	0.401	0.826 / 0.825	100
LQN(2)	9,290	3.80	0.409	0.826 / 0.827	100
LQN(16)	8,461	4.43	0.523	0.819 / 0.821	100
MLQN	9,820	3.85	0.392	0.824 / 0.826	100
MLNAQ	700	0.46	0.657	0.672 / 0.663	100
MLMoQ	590	**0.23**	0.389	0.643 / 0.649	100

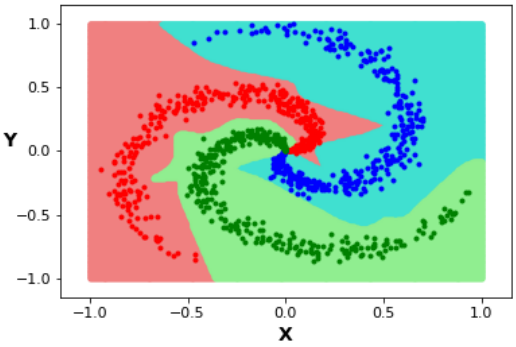

Fig. 2. The test result of 3-Spirals traind by MLMoQ.

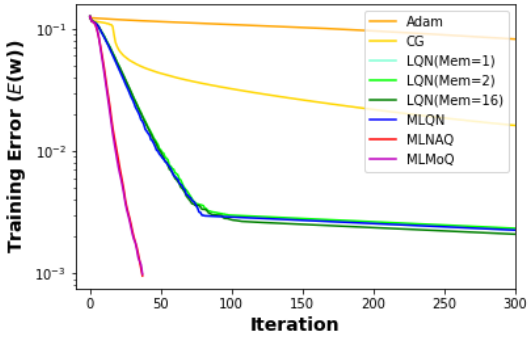

Fig. 3. The training loss vs. iteration counts of 8×8 MNIST.

whereas MLNAQ needs more time because of its disadvantage. Therefore, MLMoQ is the fastest among all algorithms used here. For measuring the accuracy of modeling, the output of the neural model trained by MLMoQ is compared with the test data in Fig.1. The figure confirmed that the test error and the model show a good match between them.

Problem 2: 3-Spirals classification problem

Next, 3-Spirals classification problem [2] is considered. For classification problems, the error function $E(\mathbf{w})$ is given by the cross-entropy (CE) error as $E_p(\mathbf{w}) = -\sum_{l=1}^{L} d_{p,l} \log(o_{p,l})$, where $d_{p,l}$ and $o_{p,l}$ denote the l^{th} elements of $\mathbf{d}_p$ and $\mathbf{o}_p$, respectively, and L is the number of output units and denotes the number of classes. Each hidden layer's neuron has the sigmoid function while the output layer uses the softmax activation function given as $softmax(z_l) = exp(z_l)/\sum_{l=1}^{L} exp(z_l)$. The inputs of the 3-Spirals problem are x and y coordinates, and the outputs are the numbers of classes $L = 3$. Therefore, the network structure is 2-10-3. Each L have 350 points and $|T_r|$ is 1,050. The median and average of $E(\mathbf{w})$, the average of computational time (sec), and the average of iteration counts (k) for ten trials are shown in Table II. The table shows that all algorithms converge to similar training errors and have a 100% convergence rate. On comparing the number of iteration counts, the QN-based algorithms converge faster than Adam and CG. Furthermore, MLNAQ and MLMoQ can obtain the solutions with extremely few iterations compared with LQN and MLQN. From the comparison between MLNAQ and MLMoQ, it is clear that MLMoQ is faster than MLNAQ. As a result, the proposed MLMoQ is the fastest among the all algorithms. In order to measure the training accuracies, the validation was conducted for the trained NN by MLMoQ. The result is shown in Fig.2. From this figure, it can be seen that the trained NN can classify 2D planes well without overfitting.

Problem 3: 8×8 MNIST

The next classification problem is 8×8 pixels MNIST handwritten digits dataset. The dataset consists of 1,797 samples which are split into 75% (1,347 samples) for the T_r and 25% (450 samples) for the test data of T_e, respectively [2]. The inputs and outputs of NN are 64 and 10, which are the numbers of the input data dimension and classes, respectively. The structure of the NN is 64-10-10. The best result of each algorithm was illustrated in Fig.3 and Fig.4, which show the graphs of training error vs. iteration counts and test accuracy vs. time, respectively. Fig.3 shows that the MLNAQ and MLMoQ have similar performance, and this is also the same for the LQN and MLQN. Adam and CG require more iteration counts to converge. As a result, it can

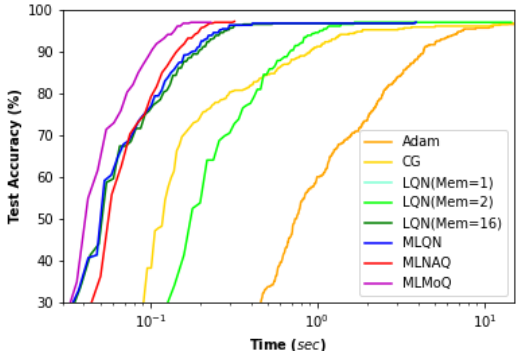

Fig. 4. The test accuracy vs. time of 8×8 MNIST.

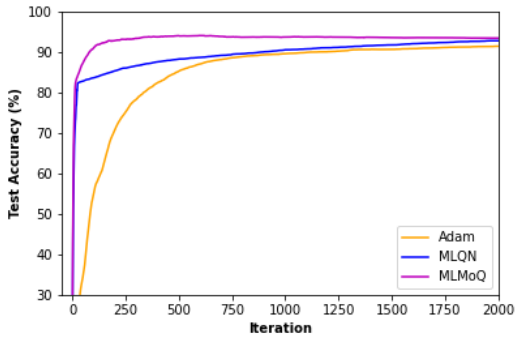

Fig. 5. The test accuracy vs. iteration counts of 28×28 MNIST.

be confirmed that the QN-based algorithms obtains a small error in the early stage of iterations compared with CG and Adam. On comparing QN-based algorithms, MLNAQ and MLMoQ converge in fewer iteration with a small training error than LQN and MLQN. On the other hand, in the computational time comparison in Fig.4, MLMoQ converges fastest among all algorithms. Therefore, it can be concluded that the proposed methods are effective for this problem.

5. Implementation of MLMoQ on Tensorflow

Finally, we evaluate the performance of the proposed algorithm for the standard 28×28 pixels MNIST dataset [1] on Tensorflow [7]. We implemented MLQN and MLMoQ on Tensorflow version 2.6.0 and simulated them in the Google Colaboratory environment [8] to validate this problem. The MLMoQ optimizer implementation in Tensorflow is made available on [9]. 28×28 dataset has $|T_r| = 60,000$ and $|T_e| = 10,000$ [1]. The inputs and outputs of NN are 784 and 10, which are the input data dimension and numbers of classes, respectively. The network structure is 784-10-10. The termination condition is set to $k_{max} = 2,000$. It is impossible to compare the exact computation time because the allocated resource changes with each run in Google Colaboratory. MLMoQ is compared with Adam and MLQN. The hyperparameters of Adam were set to default values of Tensorflow. The learning rates of MLQN and MLMoQ were set to 1.0, and the momentum coefficient is adaptively calculated [2]. The best result of each algorithm was illustrated in Fig.5, which shows the graph of test accuracy vs. iteration counts. Fig.5 shows that all algorithms reached similar test accuracy within $2,000$ iterations, but the proposed MLMoQ was able to obtain its accuracy faster. It can be concluded that the proposed MLMoQ has fast convergence property for 28×28 MNIST problem on Tensorflow.

6. Conclusion

In this paper, we studied the effectiveness of the momentum term for Memory-Less QN (MLQN) and proposed two momentum-based MLQNs, namely Memory-Less NAQ (MLNAQ) and Memory-Less MoQ (MLMoQ). The performance of the proposed methods has been demonstrated through the training for function approximation and classification problems. The proposed MLNAQ and MLMoQ showed their effectiveness in all problems. In the future, the convergence property of the momentum acceleration of the Memory-Less quasi-Newton-based methods will be clarified. Furthermore, the implementation of stochastic (mini-batch) strategies will be studied to demonstrate the effectiveness of the proposed algorithms for larger and deeper NN training.

Acknowledgments

This work was supported by the Japan Society for the Promotion of Science (JSPS), KAKENHI (17K00350 and 20K11979).

References

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.

[2] S. Mahboubi, Indrapriyadarsini, S., H. Ninomiya, and H. Asai, "Momentum acceleration of quasi-Newton based optimization technique for neural network training," *NOLTA*, vol. 12, no. 3, pp. 554–574, July 2021.

[3] H. Ninomiya, "A novel quasi-Newton optimization for neural network training incorporating Nesterov's accelerated gradient," *NOLTA*, vol. E8–N, no. 4, pp. 289–301, October 2017.

[4] J. Nocedal, and S. J. Wright, *Numerical Optimization Second Edition*, Springer Science & Business Media, 2006.

[5] D. F. Shanno, "On the convergence of a new conjugate gradient algorithm," *SIAM Journal on Numerical Analysis*, vol. 15, no. 6, pp. 1247–1257, December 1978.

[6] C. Kou, and Y. H. Dai, "A Modified Self-Scaling Memoryless Broyden-Fletcher-Goldfarb-Shanno Method for Unconstrained Optimization," *Journal of Optimization Theory and Applications* vol. 165, no. 1, pp. 209–224, April 2015.

[7] Tensorflow, https://www.tensorflow.org. (last access 17 October 2021)

[8] Google Colablatory, https://colab.research.google.com. (last access 17 October 2021)

[9] https://github.com/ninomiyalab/Memory_Less_Momentum_Quasi_Newton.